

Control de acceso mediante spring security

M6: Desarrollo de aplicaciones jee con spring framework

|AE4: Implementar mecanismos de seguridad utilizando spring security para controlar el acceso a los recursos del aplicativo

Introducción

En el mundo actual de desarrollo de aplicaciones, la **seguridad** es una preocupación primordial. Con **Spring Security**, podemos **asegurar nuestros proyectos de manera efectiva y confiable**. En esta lección, aprenderemos cómo incorporar el módulo Spring Security a nuestros proyectos y realizar una configuración básica para proteger nuestras aplicaciones.

A medida que progresamos, aprenderemos a agregar seguridad a nuestros controladores mediante anotaciones, lo que nos brindará una forma más sencilla y concisa de asegurar nuestras rutas y métodos.

También aprenderás a configurar la autenticación y autorización de usuarios, crear formularios de inicio de sesión, gestionar roles y asegurar los elementos de la capa de vista. También descubrirás cómo autenticar usuarios utilizando una base de datos y JPA.

Aprendizaje esperado

Al finalizar la lección podrás:

- Implementar mecanismos de seguridad utilizando spring security para controlar el acceso a los recursos del aplicativo.

Spring Security

Spring Security es un **marco de autenticación y control de acceso** potente y altamente personalizable. Es el estándar de facto **para proteger las aplicaciones basadas en Spring**. Es un marco que se enfoca en proporcionar autenticación y autorización a las aplicaciones Java. Como todos los proyectos de Spring, el verdadero poder de Spring Security se encuentra en la facilidad con la que se puede ampliar para cumplir con los requisitos personalizados. Estas son algunas de sus principales características:

- Soporte completo y extensible para autenticación y autorización.
- Protección contra ataques como fijación de sesión, clickjacking, falsificación de solicitudes entre sitios, etc.
- Integración de API de servlet.
- Integración con JWT.
- Integración opcional con Spring Web MVC.

Incorporando el módulo Spring Security al proyecto

Añadir Spring Security en cualquier proyecto Spring de Java es muy sencillo, sobre todo si se ha utilizado la capa de auto-configuración ya que el uso de Spring Boot simplifica enormemente las acciones de configuración que debe realizar el desarrollador para poner en marcha cualquier aplicación Spring.

Suponiendo que tienes una aplicación Spring Boot en marcha gestionado por Maven, en donde el fichero **pom.xml** tiene como parent el artefacto **spring-boot-starter-parent**, lo primero que se debe hacer es **incorporar el conjunto de dependencias provenientes de spring-boot-starter-security**. Ésta traerá de forma transitiva el resto de librerías JARs que Spring necesitará para aplicar los mecanismos de seguridad requeridos.

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-security</artifactId>
```

</dependency>

Por sólo incluir la dependencia, si el programador no realiza ninguna configuración adicional, Spring Boot de manera predeterminada **protegerá todo el acceso a la aplicación**, impidiendo que ningún usuario no identificado pueda invocar a cualquier controlador. Este mecanismo tan restrictivo suele ser suficiente para pequeñas aplicaciones donde sólo se necesita restringir el acceso de forma general, pero queda algo reducido en aplicaciones donde hay secciones accesibles y otras protegidas, dependiendo de si el usuario está identificado o de si éste tiene un rol determinado.

Configuración básica de Spring Security

Como el resto de aspectos configurables de Spring Boot, en el fichero de configuración de la aplicación, **application.properties**, hay varias propiedades que pueden ajustarse para controlar el comportamiento base de Spring Security.

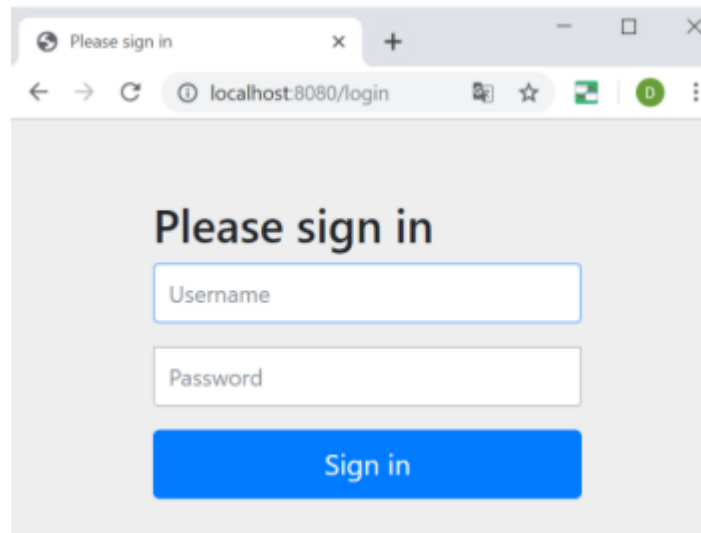
```
spring.security.user.name = user    # usuario por defecto
```

```
spring.security.user.password = user # password por defecto
```

```
spring.security.user.roles = user   # roles por defecto
```

De esta manera, sólo por incluir la dependencia, en el fichero **application.properties** puede indicarse un usuario y clave por defecto que podrá usarse para tener acceso a los controladores, y como consecuencia a las diferentes pantallas HTML de la aplicación. Cuando el usuario intente acceder a cualquier URL de la aplicación, Spring Boot y el Security Filter de HTTP redirigirá al usuario al formulario de identificación, donde solicitará al usuario a insertar el **nombre** y **password** para proceder. En ese formulario se debe indicar los valores fijados en **spring.security.user.name** y **spring.security.user.password**. Si el usuario ha introducido los valores correctos, se considera al usuario autenticado, y sin tener en cuenta el rol, dejará continuar al usuario con una navegación normal.

Con ésta configuración, si creamos un endpoint cualquiera e intentamos consumirlo, se mostrará un formulario de inicio de sesión proporcionado por Spring Security:



Si compruebas la salida por consola, se podrá ver una contraseña generada automáticamente. El nombre de usuario por defecto es «user».

```
Using generated security password: 30f20aec-fa73-4dee-a972-22f8ff77a1a5
```

Para poder realizar la configuración, debemos crear una clase que extienda de `WebSecurityConfigurerAdapter` y tiene que estar anotada como `@Configuration` y `@EnableWebSecurity`. Lo que haremos será sobrescribir el método `configure` para poder configurar nosotros el `AuthenticationManagerBuilder` que es el objeto que se encargará de realizar el manejo de la **autenticación** de usuarios.

Autenticación: verificamos la identidad del usuario.

Autorización: tipo de permisos que tiene ese usuario

En el siguiente ejemplo, tenemos creada una clase java con nombre `SecurityConfig` donde escribiremos todo el código de configuraciones de seguridad que necesitamos.

`@Configuration`

`@EnableWebSecurity`

`@EnableGlobalMethodSecurity(prePostEnabled=true)`

```
public class SecurityConfig extends WebSecurityConfigurerAdapter {
```

```
    @Autowired
```

```
    UserService usuarioServicio;
```

`@Override`

protected void `configureGlobal`(`AuthenticationManagerBuilder` auth) throws
Exception {

`auth.userDetailsService`(`usuarioServicio`);

}

}

Cómo administrar los usuarios del programa

Tener un solo usuario en la aplicación puede ser útil en ciertas ocasiones, aunque desde luego es un escenario algo limitado para aplicaciones abiertas en Internet. Por norma general se usarán varios usuarios, presumiblemente con roles de acceso diferentes para limitar las acciones que pueden realizar cada tipo de usuario.

Para ello necesitamos añadir una fuente de usuarios administrados por la aplicación y que serán usados en el proceso de autenticación por Spring Security. De esta manera, nuestra aplicación puede incorporar un mecanismo de registro de usuarios que puedan autenticarse.

Para que Spring pueda obtener la información de un usuario durante el proceso de **login**, se solicitará al conjunto de clases Java registradas en el contexto de Spring, que alguna implemente la interfaz `UserDetailsService`. Esta interfaz tiene un único método de obligada implementación que es aquel que devuelve los detalles de un usuario `UserDetails` dado un nombre de usuario, el que trata de ingresar en la aplicación. Por tanto será obligatorio implementar un bean con esa interfaz dando cuerpo a ese método.

En nuestro ejemplo, esta interfaz será implementada por la clase `UserService`, y el método a sobrescribir se llama `loadUserByUsername()`.

```
@Service
public class UserService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;
```

También debemos tener en cuenta que la interfaz UserRepository extiende de JpaRepository:

```
@Repository
public interface UserRepository extends JpaRepository<UserEntity, String> {

    @Query("SELECT u FROM UserEntity u WHERE u.email = :email")
    public UserEntity searchByEmail(@Param("email")String email);
}
```

En este repositorio hemos declarado un pequeño método personalizado de búsqueda de usuario por email (searchByEmail(String email)).

Finalmente, podemos ver como quedaría el método **loadUserByUsername** en la clase de servicio:

```
149 • @Override
150 public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
151
152     UserEntity user = userRepository.searchByEmail(email);
153
154     if (user != null) {
155
156         List<GrantedAuthority> permissions = new ArrayList<>();
157
158         GrantedAuthority p = new SimpleGrantedAuthority("ROLE_" + user.getRole().toString());
159
160         permissions.add(p);
161
162         ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
163
164         HttpSession session = attr.getRequest().getSession(true);
165
166         session.setAttribute("usuariosession", user);
167
168         return new User(user.getEmail(), user.getPassword(), permissions);
169
170     } else {
171
172         return null;
173     }
174 }
175
```

Vamos a referenciar las líneas del código para explicar el método:

152- Instanciamos un objeto del tipo *UserEntity*, haciendo uso del método *searchByEmail(email)* de la clase *UserRepository*.

154- Verificamos que el objeto Usuario no esté nulo.

156- Otorgamos **permisos** según el **rol** que tenga cada usuario. Si el usuario existe, es decir, si la búsqueda por email del repositorio nos retorna un *User*, vamos a crear una lista de permisos llamada **permissions**.

158- Creamos un objeto *GrantedAuthority* 'p' y concatenamos la palabra *ROLE_* + el rol del usuario.

160- Agregamos el objeto 'p' a la lista de permisos.

162/164- Utilizamos los atributos que nos otorga el pedido al servlet, para poder guardar la información de nuestra *HttpSession*.

168- por último, retornamos un User con su email, contraseña y permisos!

La implementación de este bean dependerá de dónde tengamos registrados los usuarios. Normalmente éstos estarán en una tabla de la base de datos, así que ya sea con SQL nativo o usando JPA con EntityManager deberemos consultar la fila asociada según el nombre de usuario. Una vez localizado sólo restará convertir o mapear ese objeto a uno esperado por Spring, del tipo UserDetails.

Creando un formulario de login

Dentro de la clase que extiende de *WebSecurityConfigurerAdapter* (en nuestro ejemplo, la clase *SecurityConfig*), vamos a sobrescribir el método *configure()*:

@Override

protected void *configure*(*HttpSecurity* http) throws Exception {

 http.authorizeRequests()

 .anyRequest().authenticated()

 .and()

 .formLogin().loginPage("/login").permitAll()

 .and()

 .logout().permitAll();}

.anyRequest().authenticated() hace referencia a que se requiere estar autenticado para todas las peticiones.

.formLogin().loginPage("/login").permitAll() refiere a que el formulario será definido en la página *login.jsp* (o *login.html*) que crearemos más adelante, las peticiones a esta página no requieren autenticación, cualquier usuario puede acceder a ella.

.logout().permitAll() activa el cierre de sesión mediante la URL */logout*, cualquier usuario puede acceder a esta URL.

Lo que prosigue es crear el formulario JSP, para que funcione correctamente debe tener dos campos: username y password, estos deben ser enviados usando el método POST a la URL `/login`.

```

<h1 class="title">Iniciar Sesión</h1>

<:url value="/login" var="loginUrl"/>

<form action="${loginUrl}" method="post">

  <c:if test="${param.error != null}">

    <p class="error">Username y password incorrectos, inténtalo nuevamente.</p>

  </c:if>

  <c:if test="${param.logout != null}">

    <p class="logout">La sesión ha sido cerrada correctamente.</p>

  </c:if>

</div>

<label for="username">Nombre</label>

<input type="text" id="username" name="username"/>

</div>

<div>

  <label for="password">Contraseña</label>

  <input type="password" id="password" name="password"/>

</div>

<button type="submit" class="btn">Log in</button>

</form>

```

Para verificar si tenemos un error en el inicio de sesión, es decir el nombre de usuario o contraseña son incorrectos verificamos si el parámetro error está presente en la URL, cuando se produce un error somos enviados a la URL: `/login?error`, creamos un mensaje de error si este parámetro existe.

```

<c:if test="${param.error != null}">

  <p class="error">Username y password incorrectos, intentalo nuevamente.</p>

```

</c:if>

De modo similar, debido a lo configurado en el método `configure`, somos enviados a la URL `/login?logout` cuando cerramos la sesión, también podemos mostrar un mensaje si lo deseamos.

Para que esto funcione recuerda que el controlador que renderiza el login debe retornar la vista `login.jsp`.

Manejo de Roles

Para comenzar, vamos a definir qué roles vamos a manejar dentro de nuestra aplicación. Hay diversas maneras de crear los roles, pero en este caso como son roles muy puntuales los manejaremos desde la aplicación en un Enum.

```
public enum Role {
    USER,
    ADMIN;
}
```

En la clase `UserEntity` ya sabemos que contamos con los atributos `username` y `password`, ahora también le agregaremos el atributo `rol`.

```
@Entity
public class UserEntity {
    @Id
    @GeneratedValue(generator = "uuid")
    @GenericGenerator(name = "uuid", strategy = "uuid2")
    private String id;

    private String username;

    private String password;

    @Enumerated(EnumType.STRING) ←
    private Role role;

    private String email;
```

Vamos a adaptar el método `configure` de nuestra clase de seguridad para indicar el comportamiento que debe tener nuestra aplicación según el rol que tenga el usuario.

```

@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
        .antMatchers("/admin/*").hasRole("ADMIN")
        .antMatchers("/css/*", "/js/*", "/img/*", "/*")
        .permitAll()
        .and().formLogin()
        .loginPage("/login")
        .loginProcessingUrl("/logincheck")
        .usernameParameter("email")
        .passwordParameter("password")
        .defaultSuccessUrl("/start")
        .permitAll()
        .and().logout()
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login")
        .permitAll()
        .and().csrf()
        .disable();
}

```

`.antMatchers("/admin/*").hasRole("ADMIN")`: indica que a todos los endpoints que nazcan desde /admin sólo podrán acceder aquellos usuarios que tengan el Rol ADMIN asignado.

`.antMatchers("/css/*", "/js/*", "/img/*", "/*").permitAll()` : indica que todos aquellos archivos que provengan de las carpetas indicadas entre paréntesis, serán permitidos para todos los usuarios (estilos, animaciones, imágenes, etc)

A continuación tenemos las propiedades de **login y logout**, que también podrán ser accedidas por todos los usuarios.

En las líneas que indican los parámetros `.usernameParameter("email")` y `.passwordParameter("password")`, debemos recordar que el parámetro entre paréntesis deben tener el mismo nombre que el atributo a evaluar.

En este caso, el “nombre de usuario” o **username** está determinado por el atributo **email** del User.

Añadiendo seguridad a los elementos de la vista

Spring Security cuenta con **taglib** que nos provee acceso a la información de seguridad de los usuarios y posibilita el filtrado del contenido que se muestra al usuario en base a sus privilegios. Para utilizar taglib debemos agregar la siguiente dependencia:

```
<dependency>

  <groupId>org.springframework.security</groupId>

  <artifactId>spring-security-taglibs</artifactId>

  <version>${security.version}</version>

</dependency>
```

Declaramos el taglib en el archivo **JSP** de la siguiente manera:

```
<%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
```

La etiqueta **authorize** nos sirve para filtrar el contenido que se mostrará en base a los privilegios del usuario, por ejemplo, podemos definir un contenido que solo será mostrado a aquellos usuarios con privilegios de administrador, esta etiqueta la utilizaremos de la siguiente manera:

```
<sec:authorize access="hasRole('ROLE_ADMIN')">
```

Este contenido se muestra si el usuario tiene el privilegio:
ROLE_ADMIN.

```
</sec:authorize>
```

Estableciendo el atributo **access="isAuthenticated()"** mostraremos contenido a todos los usuarios autenticados, la expresión puede ser más compleja si utilizamos SpEL, por ejemplo, la expresión: **access="isAuthenticated() and principal.username == 'jesus'"**.

Otra forma de usar esta etiqueta es indicando el atributo url, por medio del mismo, el contenido será generado si el usuario actual tiene permiso para acceder a la URL indicada.

```
<sec:authorize url="/user/delete">
```

```
</sec:authorize>
```

Si deseamos obtener el nombre del usuario y sus roles, usaremos las siguientes etiquetas:

```
<sec:authentication property="principal.username" />
```

```
<sec:authentication property="principal.authorities" />
```

Obteniendo el usuario autenticado en el controlador

Si no estás utilizando Thymeleaf en tu página JSP y deseas obtener el usuario actual desde el controlador, puedes hacerlo utilizando la clase `SecurityContextHolder` de Spring Security.

Primero debes tener los imports necesarios:

```
import org.springframework.security.core.Authentication;
```

```
import org.springframework.security.core.context.SecurityContextHolder;
```

```
import org.springframework.security.core.userdetails.UserDetails;
```

En el método del controlador donde deseas obtener el usuario actual, puedes hacer lo siguiente:

```
@GetMapping("/ejemplo")
public String ejemplo(Model model) {
    // Obtener el objeto de autenticación actual
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    // Verificar si el usuario está autenticado
    if (authentication.isAuthenticated()) {
        // Obtener los detalles del usuario
        Object principal = authentication.getPrincipal();
        String username;

        if (principal instanceof UserDetails) {
            username = ((UserDetails) principal).getUsername();
        } else {
            username = principal.toString();
        }

        // Pasar el nombre de usuario al modelo para utilizarlo en la vista
        model.addAttribute("username", username);
    }

    // Resto de la lógica del controlador
    return "ejemplo";
}
```

En este ejemplo, para obtener el objeto de autenticación actual utilizamos `SecurityContextHolder.getContext().getAuthentication()`. Luego, verificamos si el usuario está autenticado y obtenemos los detalles del usuario, que pueden ser un objeto `UserDetails` o simplemente una cadena que representa el nombre de usuario.

Agregando seguridad en los controladores mediante anotaciones

Spring Security nos permite hacer uso de anotaciones que restringen los accesos a nivel método. Normalmente estas anotaciones se presentan en los componentes de tipo controlador, para poder manejar el nivel de acceso a cada endpoint. Aquí veamos un ejemplo simple:

```
@Controller
public class RoleController {
    @GetMapping("group1")

    @PreAuthorize("hasRole('ROLE_group1')")
    public String group1() {
        return "group1 message";
    }

    @GetMapping("group2")

    @PreAuthorize("hasRole('ROLE_group2')")
    public String group2() {
        return "group2 message";
    }
}
```

En este caso podemos ver la anotación `@PreAuthorize`. En el primer caso, podemos ver como pre-autoriza el acceso a varios miembros determinados en un grupo.

También puede pre-autorizar el acceso al endpoint para cualquier rol declarado:

```
@PreAuthorize("hasAnyRole('ROLE_USER', 'ROLE_ADMIN')")
```

La anotación `@PreAuthorize` verifica la expresión dada antes de ingresar el método, mientras que la anotación `@PostAuthorize` la verifica después de la ejecución del método y podría alterar el resultado.

Tanto las anotaciones `@PreAuthorize` como `@PostAuthorize` proporcionan un control de acceso basado en expresiones. Entonces, los predicados se pueden escribir usando SpEL (Spring Expression Language).

Autenticación y encriptación de contraseñas mediante JPA

Un mecanismo de seguridad que hay que tener en cuenta a la hora de guardar los datos del usuario es **proteger las contraseñas**, usualmente no se almacena la contraseña como tal, lo que guardamos en la base de datos es el valor hash de las mismas, de este modo si alguien logra tener acceso a nuestra base de datos de usuarios no podrá leer las contraseñas.

Para esta tarea Spring Security utiliza la interfaz `PasswordEncoder`, podemos crear nuestra propia implementación o utilizar una de las proporcionadas por el Framework, para este ejemplo utilizaremos la clase `BCryptPasswordEncoder`.

En el método `configureGlobal` de la clase de seguridad, teníamos el objeto `AuthenticationManagerBuilder` que se encarga de realizar el manejo de la autenticación de usuarios. A continuación del llamado a `userDetailsService` para autenticar, colocamos otro punto para llamar al encoder.

`@Autowired`

```
Public void configureGlobal(AuthenticationManagerBuilder auth) throws
exception{
```

```
auth.userDetailsService(usuarioService)
```

```
    .passwordEncoder(new BCryptPasswordEncoder());
```

```
}
```

Una vez definido el método de autenticación y encriptación de contraseñas, solo nos resta setear el encoder al momento de guardar la contraseña generada por el usuario. Es necesario que en la base de datos quede persistida la contraseña ya codificada. Esto lo realiza JPA con referencia de las anotaciones ya mencionadas.

Para poder definir la encriptación al momento de la creación de la contraseña, vamos a agregar el llamado al encoder en el método de creación de usuario en el servicio. Por ejemplo:

@Transactional

```
public void create(String name,String email, String password){  
  
    UserEntity user = new UserEntity();  
  
    user.setName(name);  
  
    user.setEmail(email);  
  
    user.setPassword(new BCryptPasswordEncoder().encode(password))
```

De esta manera estamos pidiéndole a JPA que **codifique la contraseña en el mismo momento en el que se setee en la variable password**. Si realizamos una consulta en la base de datos, podremos ver que la contraseña está compuesta de caracteres alfanuméricos en cadena larga.

Por ejemplo, si al momento de crear el usuario colocamos como contraseña la cadena "123456", la clase **BCryptPasswordEncoder**, generará algo como esto:

\$2a\$10\$fYoIYBQcszFwBxwT3hbj.QY6TDz1SvDDi9SQgY028maC6khfg2di

y ese será el valor **hash** con el que queda guardada en la base de datos.

Cierre

Durante este recorrido, has aprendido a fortalecer la seguridad de tus proyectos incorporando Spring Security, permitiéndote configurar de manera efectiva la autenticación y autorización de usuarios. Has adquirido conocimientos clave para crear formularios de inicio de sesión, gestionar roles y asegurar los elementos de la capa de vista.

Además, exploramos cómo autenticar usuarios utilizando una base de datos y JPA, lo que te brinda herramientas poderosas para proteger tus aplicaciones y datos de manera confiable.

¡Nos vemos de nuevo, más adelante! 

Referencias

- **Arteco Consulting. (s. f.).** Tutorial Spring Boot (Quick Start para programar con Java/Spring Boot). Arteco Consulting. Recuperado de <https://www.arteco-consulting.com/articulos/tutorial-springboot/>
- **Desarrollo Avanzado Web. (2017, 2 de junio).** Spring Security Formulario de Login JSP. Acodigotutoriales. Recuperado de <https://acodigo.blogspot.com/2017/06/spring-security-formulario-de-login-jsp.html>
- **Baeldung. (s. f.).** Security with Spring. Baeldung. Recuperado de <https://www.baeldung.com/security-spring>

¡Muchas gracias!

Nos vemos en la próxima lección

